

# State Management

React Router fundamentally changes how you think about client-side state. Its `loader / action` pattern handles the most common use case for global state management libraries, such as [Redux](#), [React Query](#), or [Zustand](#): fetching server data and keeping it synchronized after mutations. Since React Router manages this natively, you typically don't need a separate library for such *remote state*.

**What this means in practice:** A server-centric model replaces the classic client-side fetch-on-mount, store-in-state, manually-invalidate cycle:

1. Route enters, `loader()` runs, and then data flows into the component. For example, data flows via the `useLoaderData()` hook.
2. User submits form, `action()` runs, and then React Router revalidates all loaders automatically.

## Contents

### State Types and Their Native Tools

Server State via `loader()` and `action()`

Mutations via `action()` and `Form`

Component-Local Mutations via `useFetcher()`

Cross-Route Data via `useRouteLoaderData()`

Middleware Context

Navigation State

URL State

Persistent State via Cookies and Sessions

Optimistic State

Error State

Native React State Primitives

## State Types and Their Native Tools

| State Type                                | Description                                 | Native Tool                                         |
|-------------------------------------------|---------------------------------------------|-----------------------------------------------------|
| <a href="#">Server / Remote State</a>     | Data fetched from a server                  | <code>loader</code> + <code>useLoaderData</code>    |
| <a href="#">Mutations</a>                 | Write operations to the server              | <code>action</code> + <code>Form</code>             |
| <a href="#">Component-Local Mutations</a> | Mutations without navigation                | <code>useFetcher</code>                             |
| <a href="#">Cross-Route Data</a>          | Accessing another route's loader data       | <code>useRouteLoaderData</code>                     |
| <a href="#">Middleware Context</a>        | Request-scoped data injected before loaders | <code>context.set</code> / <code>context.get</code> |



| progress         |                                             |                                                                                         |
|------------------|---------------------------------------------|-----------------------------------------------------------------------------------------|
| URL State        | State encoded in the URL                    | <code>useSearchParams</code> , <code>use</code>                                         |
| Persistent State | State persisted across requests via cookies | <code>createCookie</code> , <code>createCookieSession</code>                            |
| Optimistic State | Temporary UI before server confirms         | <code>fetcher.formData</code> , <code>useNavigation</code> , <code>useOptimistic</code> |
| Error State      | Loader/action errors per route              | <code>useRouteError</code>                                                              |
| React Primitives | Native React state management primitives    | <code>useState</code> , <code>useMemo</code> , <code>C</code> API, and so on            |

## Server State via `loader()` and `action()`

This is the central state management pattern in React Router. It replaces client-side data fetching patterns, such as `fetch-on-mount` and manual cache invalidation, with a declarative, server-centric model.

- A `loader` runs on the server before the route renders. Its return value becomes the component's data via `useLoaderData()`.
- An `action` handles mutations. After it completes, React Router automatically revalidates all active loaders. React Router manages the data lifecycle entirely.

For `loader`, `data classification` (critical vs. non-critical), streaming patterns, `action processing`, and `resource routes`, see [Data Fetching](#).

## Mutations via `action()` and Form

The `<Form>` component serializes form state into `FormData` and submits it to the route's `action`. After the action completes, React Router revalidates all active loaders automatically. This is the primary mutation pattern when the operation *should trigger a navigation* or a full-page state update.



## Component Local Mutations via useFetcher()

`useFetcher` [↗](#) triggers loaders and actions **without causing navigation**. It gives each component its own mutation lifecycle, making it suitable for inline edits, autosave, toggling, and any mutation that shouldn't change the URL.

From a state management perspective, each fetcher exposes its own lifecycle. `fetcher.state` exposes the mutation lifecycle while `fetcher.data` holds the action's return value. This enables per-component feedback (for example, inline validation errors) without global state.

For fetcher usage patterns and resource routes, see [Data Fetching](#). For visual feedback patterns based on `fetcher.state`, see [Loading States](#).

## Cross-Route Data via useRouteLoaderData()

`useRouteLoaderData` [↗](#) accesses the loader data of any currently active route by its route ID. This avoids prop-drilling when a child route needs data that a parent or sibling route already loaded.

A common use case is accessing root-level data, such as session, locale, or site configuration, from deeply nested routes without passing it through every intermediate layout. The route ID passed to `useRouteLoaderData` must match the route's `id` as defined in the route configuration. The hook returns `undefined` if the target route isn't currently active.

```
1 // routes/root.tsx
2 export async function loader() {
3   const user = await getUser();
4   const locale = await getLocale();
5   return { user, locale };
6 }
7
8 // routes/products/product-detail.tsx
9 import { useRouteLoaderData } from "react-router-dom";
10
11 export default function ProductDetail() {
12   const rootData = useRouteLoaderData < type...
13   return <p>Welcome, {rootData?.user?.name}</p>;
14 }
```

## Middleware Context



for loaders to independently resolve cross-cutting concerns such as sessions, app configuration, or API clients.

For middleware definition, pipeline ordering, and writing middleware functions, see [Data Fetching](#).

**Note**

Middleware context isn't the React Context API. While middleware context carries server-side data, it serves a distinct role from loaders. Middleware context operates in the request lifecycle (before render), is consumed by loaders and actions, and is scoped to a single request. The React [Context API](#) operates in the render lifecycle and is consumed by components. Data from middleware context typically reaches components via the `loader` return value and `useLoaderData`.

## Navigation State

`useNavigation` [↗](#) exposes the global navigation state, useful for pending UI during route transitions.

```
1 import { useNavigation } from "react-router";
2
3 function GlobalSpinner() {
4   const navigation = useNavigation();
5   if (navigation.state === "idle") {
6     return null;
7   }
8   return <Spinner />;
9 }
```

`navigation.formData` is available during form submissions, enabling optimistic UI based on the submitted values before the server responds. For loading state patterns in detail, see [Loading States](#).

## URL State

The URL is a state container. Search parameters, path parameters, and the location object are reactive state sources that survive page refreshes, are shareable via links, and integrate with the browser's history stack.

### `useSearchParams()`

`useSearchParams` [↗](#) reads and writes URL query parameters. It replaces `useState` for state that should



URL, or be accessible to loaders (via `request.url`), it belongs in search params.

```
1 import { useSearchParams } from "react-router-dom";
2
3 function ProductFilters() {
4   const [searchParams, setSearchParams] = useSearchParams();
5   const category = searchParams.get("category");
6
7   return (
8     <select value={category} onChange={(e) => {
9       <option value="all">All</option>
10      <option value="shoes">Shoes</option>
11      <option value="bags">Bags</option>
12    }>
13   );
14 }
```

## useParams()

`useParams` [↗](#) returns the dynamic path parameters for the current route. It's read-only and reactive, the component re-renders when the route changes.

```
1 import { useParams } from "react-router";
2
3 function ProductDetail() {
4   const { productId } = useParams();
5   // productId is derived from the URL, e.g., "/products/123"
6 }
```

## Persistent State via Cookies and Sessions

Sometimes, state must survive page refreshes and revisits without belonging to the URL. Examples include user preferences (theme, locale, dismissed banners), shopping cart identifiers, and authentication tokens. Cookies and server sessions are React Router's native mechanism for this kind of persistent state.

Unlike `useState` (transient) or `useSearchParams` (URL-visible), cookies travel with every HTTP request and are available in loaders and actions on the server. This behavior makes cookies a single source of truth that requires no client-side synchronization, avoids `localStorage` SSR issues, and works even before scripts load.

React Router provides `createCookie` [↗](#) for simple key-value cookies and `createCookieSessionStorage` [↗](#) for structured, typed session data. Both integrate directly with the `loader` / `action` lifecycle: cookies are



## URL State

| Criterion                                        | Cookies<br>( <code>createCookie</code> ) | Sessions<br>( <code>createCookieSess:</code> |
|--------------------------------------------------|------------------------------------------|----------------------------------------------|
| Survives<br>page<br>refresh                      | Yes                                      | Yes                                          |
| Shareable<br>via link                            | No                                       | No                                           |
| Available<br>in loaders                          | Yes                                      | Yes                                          |
| Works<br>without JS                              | Yes                                      | Yes                                          |
| Structured<br>/ typed<br>data                    | Manual parsing                           | Built-in typed API                           |
| Security<br>( <code>httpOnly</code> ,<br>signed) | Yes                                      | Yes (with secrets)                           |
| Best for                                         | Simple<br>preferences, flags             | Auth, multi-field user s                     |

For details on cookies and sessions APIs, see [Data Fetching](#).

## Optimistic State

Optimistic UI updates the interface immediately, before the server confirms the mutation, and rolls back if the operation fails. React Router and React offer complementary approaches, each suited to a different kind of state.

### Via `fetcher.formData` (React Router)

`useFetcher` exposes the submitted `formData` while an action is in flight. This enables optimistic reads directly from the pending submission without additional state management. This pattern requires no extra hooks and works with any version of React. It's the idiomatic React Router approach for simple optimistic updates derived from the submitted form values.



```
1 const fetcher = useFetcher();  
2
```



```
8   <button type= submit >{iscomplete ? '✓' : '✗'}</button>
9   </fetcher.Form>
10  );
```

## Via `useNavigation()` (React Router, URL-Driven State)

When state is serialized into URL search parameters and updates trigger a `navigate()` call, `useNavigation()` provides a navigation-aware optimistic path. While a navigation is pending, `navigation.location` holds the target `Location` object. Reading the intended query parameters from that target reflects the user's action immediately, without any local state to manage.

```
1  import { useNavigation } from "react-router";
2
3  function Filters({ items }) {
4    const navigation = useNavigation();
5
6    // While a navigation is pending, read from
7    // Once it settles, fall back to the server
8    const effectiveItems = navigation.location
9      ? new URLSearchParams(navigation.location.search)
10     : items;
11
12    return (
13      <ul>
14        {effectiveItems.map((item) => (
15          <li key={item}>{item}</li>
16        ))}
17      </ul>
18    );
19  }
```

This works because `navigation.location` is only defined while a navigation is in flight. It holds the destination URL, so the component can derive the optimistic state from the search params the router is navigating to. When the navigation completes, `navigation.location` becomes `undefined` in the same render cycle that delivers the fresh loader props. The ternary falls through to the new server data automatically.

Key properties of this pattern:

- No local state: Nothing to sync, reset, or reconcile.
- Survives the pending phase: The optimistic value is read from the URL the router is navigating to, so it persists for the entire loader fetch.
- Self-cleaning: Once the navigation settles, the component naturally falls back to the new props.



mutations that don't change the URL, prefer `fetcher.formData` or `useOptimistic` instead.

## Via `useOptimistic()` (React)

`useOptimistic` [↗](#) applies a temporary state update that React automatically reverts once the underlying async operation settles. It's suited for more complex optimistic transformations where the optimistic state isn't a direct read from `formData` or the URL, for example, inserting a new item into a list.

```
1 import { useOptimistic } from 'react';
2
3 function ProductList({ products }: { products: Product[] }) {
4   const [optimisticProducts, addOptimistic] = useOptimistic(
5     products,
6     (state, newProduct: Product) => [...state, newProduct]
7   );
8
9   async function handleAdd(formData: FormData) {
10     const name = formData.get('name') as string;
11     addOptimistic({ id: 'temp', name }); // Add to optimistic state
12     await createProduct(name);          // Actual async operation
13   }
14
15   return (
16     <>
17       <form action={handleAdd}>
18         <input name="name" />
19         <button type="submit">Add</button>
20       </form>
21       <ul>
22         {optimisticProducts.map(p => (
23           <li key={p.id} style={{ opacity: p.id === 'temp' ? 0.5 : 1 }}>
24             {p.name}
25           </li>
26         ))}
27       </ul>
28     </>
29   );
30 }
```

## Error State

React Router treats errors from loaders and actions as a first-class route state. When a loader or action throws an error, React Router catches the error and renders the nearest `ErrorBoundary` [↗](#) export in the route hierarchy.

`useRouteError` [↗](#) accesses the thrown error inside an `ErrorBoundary`. `isRouteErrorResponse` [↗](#) distinguishes between Response-based errors (thrown via `throw new Response(...)` or `data()` [↗](#) with a





```
1 // routes/products.tsx
2 import { useRouteError, isRouteErrorResponse }
3
4 export function ErrorBoundary() {
5   const error = useRouteError();
6
7   if (isRouteErrorResponse(error)) {
8     return (
9       <div>
10         <h1>{error.status}</h1>
11         <p>{error.statusText}</p>
12       </div>
13     );
14   }
15
16   return <div>An unexpected error occurred.</div>;
17 }
```

## Native React State Primitives

The sections above cover state that React Router manages as part of the routing and data lifecycle. What remains to complete the picture, is state that lives entirely on the client, managed by React's own primitives. These are the built-in hooks and patterns for local component state, derived values, cross-component sharing, and integration with external stores.

### Local UI State

#### `useState()`

The baseline primitive for synchronous, isolated state within a single component. Use the [useState](#)  hook when exactly one component owns the state and you don't need to share it.

```
1 function Counter() {
2   const [count, setCount] = useState(0);
3   return <button onClick={() => setCount((c)
4 }
```

#### `useReducer()`

The [useReducer](#)  hook is preferable over `useState` when state transitions are complex, interdependent, or benefit from explicit action semantics.

```
1
```



```
6     case "increment":
7       return { ...state, count: state.count + 1 }
8     case "setStep":
9       return { ...state, step: action.payload }
10  }
11 }
12
13 function Counter() {
14   const [state, dispatch] = useReducer(reducer, { count: 0, step: 1 })
15   return (
16     <>
17       <p>Count: {state.count}</p>
18       <button onClick={() => dispatch({ type: "increment" })}>Increment</button>
19       <input
20         type="number"
21         value={state.step}
22         onChange={(e) => dispatch({ type: "setStep", payload: e.target.value })}
23       />
24     </>
25   );
26 }
```

## Derived Local UI State

Derived state is any value you compute from existing state or props rather than store independently. It's a pure function of other state and has no source of truth of its own.

**Key rule:** Don't store derived values in `useState`. This practice creates a secondary source of truth and forces manual synchronization, which is a common source of bugs.

```
1 // Bad: redundant state, sync required
2 const [fullName, setFullName] = useState(`${firstName} ${lastName}`);
3
4 // Good: computed inline, always in sync
5 const fullName = `${firstName} ${lastName}`;
```

### `useMemo()`

`useMemo` [↗](#) caches the result of an expensive computation and only recalculates it when one of its dependencies changes, preventing unnecessary recalculation on every render. Use this for expensive transformations, such as sorting, filtering, and aggregating, over arrays or objects.

**When to use `useMemo`:** For cheap computations, a plain variable in the render body is sufficient. Reserve `useMemo` for expensive computations or when referential stability is required (for example, as a dependency in `useEffect` or `memo`).





## useCallback()

`useCallback` [↗](#) memoizes a function reference.

Without it, every render creates a new function instance, which breaks referential equality checks in child components wrapped with `memo` [↗](#).

```
1 const handleDelete = useCallback((id: string)
2   setProducts((prev) => prev.filter((p) => p.
3 }, []); // stable reference – no unnecessary
```

**When `useCallback` isn't needed:** If `memo` doesn't wrap the receiving component, `useCallback` has no observable effect and only adds noise. Apply it specifically when passing callbacks to memoized components or as dependencies of other hooks (`useEffect`, `useMemo`) where referential stability matters.

## Shared UI State

`Context` [↗](#) propagates state through the React tree without prop drilling. It's appropriate for state that changes infrequently and must be accessible across many components.

### Note

For request-scoped data that loaders and actions need (session, auth, and config), see [Middleware Context](#). The React Context API is for UI state shared across the component tree during rendering.

**Good use cases:** Locale, user preferences, and feature flags.

**Poor use cases:** High-frequency updates (for example, mouse position, real-time data), large state objects where many consumers only need a small slice.

### Basic Setup

```
1 // preferences-context.tsx
2 type PreferencesState = {
3   density: "compact" | "comfortable",
4   setDensity: (d: "compact" | "comfortable")
5 };
6 const PreferencesContext = (createContext < P
7
8 export function PreferencesProvider({ childre
9   const [density, setDensity] = (useState < '
```



```
15   };
16 }
17
18 export function usePreferences() {
19   const ctx = useContext(PreferencesContext);
20   if (!ctx) {
21     throw new Error("usePreferences must be used within a PreferencesContext");
22   }
23   return ctx;
24 }
```

#### Note

Every context value change re-renders all its consumers, regardless of whether they use the changed part of the value. Mitigate this by splitting contexts. A component using, for example, only `PreferencesContext` doesn't re-render when `user` changes.



```
1 // Instead of one large context:
2 <AppContext.Provider value={{ user, preferences }} />
3
4 // Split into focused contexts:
5 <UserContext.Provider value={user} />
6 <PreferencesContext.Provider value={preferences} />
7 <LocaleContext.Provider value={locale} />
```

## Context Selector Pattern

When a context holds multiple values but a consumer only needs one slice, splitting contexts isn't always practical. Domain stores often have many fields and many consumers each needing different slices. The selector pattern addresses this demand by accepting a selector function and returning only the selected slice. Combined with [useSyncExternalStore](#), subscriptions are per-slice: a consumer re-renders only when its slice changes.

The context holds a reference to an external store, not the state itself. The hook drives the subscription via `useSyncExternalStore`:



```
1 const StoreLocatorContext = createContext<StoreLocator>();
2
3 export function useStoreLocator<T>(selector: (store: StoreLocator) => T) {
4   const store = useContext(StoreLocatorContext);
5   if (!store) {
6     throw new Error('useStoreLocator must be used within a StoreLocatorContext');
7   }
8   const getSnapshot = useCallback(() => selector(store))
9   return useSyncExternalStore(store.subscribe, getSnapshot);
10 }
```



```
1 function StoreLocatorBadge() {
2   const isOpen = useStoreLocator((s) => s.isOpen)
3   const close = useStoreLocator((s) => s.close)
4   // ...
5 }
6
7 function StoreLocatorForm() {
8   const config = useStoreLocator((s) => s.config)
9   // ...
10 }
```

#### Note

React's `useContext` doesn't support selectors natively—every consumer re-renders on any context value change. A selector hook that only wraps `useContext` is a convention and doesn't, by itself, skip renders. Actual render memoization requires either `useSyncExternalStore`, as shown earlier, which is tearing-safe and concurrent-mode correct, or `React.memo` on the consuming component, combined with a referentially stable selector result. For high-frequency updates on large domain stores, prefer the `useSyncExternalStore` approach.

## Global Client State via `useSyncExternalStore()`

`useSyncExternalStore` [↗](#) integrates state that lives **outside the React tree**—vanilla JavaScript objects, browser APIs, or custom event emitters. It is concurrent-mode safe and does not require a Provider.

### Building a Minimal Store

```
1 // store.ts
2 type Listener = () => void;
3
4 function createStore<T>(initialState: T) {
5   let state = initialState;
6   const listeners = new Set<Listener>();
7
8   return {
9     getSnapshot: () => state,
10    subscribe: (listener: Listener) => {
11      listeners.add(listener);
12      return () => listeners.delete(listener);
13    },
14    setState: (updater: (prev: T) => T) => {
15      state = updater(state);
16      listeners.forEach((fn) => fn());
17    },
18  };
19 }
```

[Try for free](#)

```
1 // useCart.ts
2 import { useSyncExternalStore } from "react";
3 import { cartStore } from "../store";
4
5 export function useCart() {
6   const state = useSyncExternalStore(cartStore);
7
8   return {
9     items: state.items,
10    addItem: (item: CartItem) => cartStore.setItem(item),
11  };
12 }
```

Every component calling `useCart()` subscribes automatically. No Provider, no wrapping component required.

### Wrapping Browser APIs

The same pattern makes any browser API reactive:

```
1 // useOnlineStatus.ts
2 const subscribe = (cb: () => void) => {
3   window.addEventListener("online", cb);
4   window.addEventListener("offline", cb);
5   return () => {
6     window.removeEventListener("online", cb);
7     window.removeEventListener("offline", cb);
8   };
9 };
10
11 export const useOnlineStatus = () =>
12   useSyncExternalStore(
13     subscribe,
14     () => navigator.onLine,
15     () => true, // server snapshot (SSR)
16   );
```

The same pattern applies to `matchMedia`, `localStorage`, `BroadcastChannel`, `visibilitychange`, and similar APIs.

### Critical: getSnapshot Stability

`getSnapshot` must return a **referentially stable value** when state hasn't changed. Returning a new object on every call causes an infinite render loop.

```
1 // WRONG – new object reference on every call
2 getSnapshot: () => ({ ...state });
3
4 // CORRECT – same reference when state is unchanged
5 getSnapshot: () => state;
```

[Try for free](#)

| Criterion             | <code>useSyncExternalStore</code> | Context API                             |
|-----------------------|-----------------------------------|-----------------------------------------|
| State location        | Outside React (module scope)      | Inside React tree                       |
| Re-render granularity | Only subscribed components        | All consumers of the context            |
| Provider required     | No                                | Yes                                     |
| Concurrent Mode safe  | Yes (tearing-safe by design)      | Partially                               |
| Best for              | High-frequency or global state    | Infrequently changing, tree-local state |

## Additional State Primitives

### `use(Promise)`

Reads a Promise directly inside a component and integrates with `Suspense`. Eliminates the need for `useEffect`-based data fetching and loading state management. For declarative loading patterns with `use()` and `Suspense`, see [Loading States](#).

```
1 import { use, Suspense } from "react";
2
3 function UserProfile({ userPromise }: { userPromise: Promise<User> }) {
4   const user = use(userPromise); // Suspends
5   return <p>{user.name}</p>;
6 }
7
8 function App() {
9   const userPromise = fetchUser(userId);
10  return (
11    <Suspense fallback=<Skeleton />>
12      <UserProfile userPromise={userPromise} />
13    </Suspense>
14  );
15 }
```

### `useTransition()` and `useDeferredValue()`

`useTransition` marks state updates as non-urgent, keeping the UI responsive during heavy re-renders.

```
1
2
```



```
6   }));  
7 }
```

`useDeferredValue` defers the propagation of a rapidly changing value, similar to debouncing but integrated with React's concurrent scheduler.



```
1 const deferredQuery = useDeferredValue(searchQuery)  
2 // deferredQuery lags behind searchQuery during updates  
3 const results = useMemo(() => search(deferredQuery), [deferredQuery])
```

### `useActionState()`

`useActionState` [↗](#) manages the full lifecycle of an async action: pending status, return value, and error state. It replaces manual `useState` and `useTransition` wiring around actions. Listed here for completeness as a React primitive.



#### Note

In the React Router framework, `useActionState` has no practical relevance. Route `action` + `Form` and `useFetcher` provide the same pending or error lifecycle and additionally trigger automatic loader revalidation after mutations.

`useActionState` bypasses React Router's data lifecycle entirely, loaders aren't revalidated after a `useActionState` action completes. Even for purely client-side logic, a `clientAction` with `useFetcher` is preferable because it keeps the component within React Router's consistent data flow and avoids introducing a parallel mutation pattern.

DID THIS ARTICLE SOLVE YOUR  
ISSUE?

Let us know so we can improve!

[Share your feedback](#)





Try for free



Commerce Cloud

Lightning Design System

Einstein

Quip

Trailhead

Sample Apps

AppExchange

Blog

Salesforce Admins

Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved.](#) Various trademarks held by their respective owners. Salesforce, Inc.  
Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#)

[Terms of Service](#)

[Privacy Information](#)

[Responsible Disclosure](#)

[Trust](#)

[Contact](#)

[Use of Cookies](#)

[Cookie Preferences](#)



[Your Privacy Choices](#)





